Part-4

Securing Apache

This series of articles addresses the Web security concerns of information security experts, systems administrators and all those who want to jump-start their careers in this domain. In this issue, we will delve deeper into Web application security, peel off the layers of cross-site tracing (XST) and cross-site history manipulation (XSHM) attacks, and recommend some solutions.

In Part 2 of this series, we discussed XSS attacks. And among the solutions to the problem, we talked about using *HttpOnly* cookie mechanisms. *HttpOnly* is a session protection mechanism that specifies that the session identifier should not be accessed from the application DOM. In that case, the attacker cannot hijack the session using malicious scripts, because *document.cookie* does not return anything useful. *HttpOnly* works with almost all modern browsers. It is implemented simply as follows:

```
Set-Cookie: PHPSESSIONID=[token]; HttpOnly
```

This was, by far, one of the best ways to stop XSS attacks from fetching a victim's cookies—until it was found that the Web server *HTTP TRACE* method (more about it later) can be used to bypass *HttpOnly* security mechanisms. If an attacker forces the victim's browser using XSS to issue a *TRACE* request to the Web server. And if this browser has a cookie for that domain, the cookie will be automatically included in the request headers, and will therefore be echoed back in the resulting response. At that point, the cookie string will be accessible by JavaScript, and it will be finally possible to send it to a third party even when the cookie is tagged as *HttpOnly*. Hence, XST is nothing but an XSS attack using *TRACE*.

HTTP TRACE and XST

The *HTTP TRACE* request is a method designed for debugging problems such as network connection errors between servers. It is defined with other well-known methods like *GET, PUT, DELETE*, etc. When a client sends a *TRACE* request to a compliant server, the server responds by echoing back the header sent by the client. An attacker can exploit this to circumvent *HTTPOnly* cookies by injecting code that sends an asynchronous *XMLHttpRequest* with the *TRACE* method, and receiving the *HTTPOnly* cookie in